Lab: File System-Based Digital Forensics

Case Background: FAT32 USB Investigation

As a **Junior Digital Forensic Analyst** at **Sentinel Security Solutions**, you have been assigned to investigate a **FAT32 USB drive** that may contain evidence of deleted or timestamp-manipulated files. The forensic team has created a forensic disk image (fat32_forensic.dd), and your role is to:

1. Analyze the file system metadata to detect anomalies.
2. Identify and analyze suspicious timestamps.
3. Recover deleted files using CLI forensic tools.
4. Extract multiple files using a Python script.
5. Compare NTFS and FAT32 file system behaviors.
6. Validate file integrity using forensic hashing techniques.

This lab builds upon the Technical Lesson, extending your skills toward forensic automation in the next module.

Completing this lab will allow you to demonstrate your ability to:

- Use Sleuth Kit tools (fls, istat, icat, fsstat) to recover deleted files
- Calculate byte offsets from file metadata for FAT32 recovery
- Automate file carving using Python and a custom offsets file
- Detect suspicious timestamps that may indicate tampering
- Compare NTFS and FAT32 behavior in forensic recovery contexts
- Validate recovered file integrity using MD5 hashing

Tools and Resources

Tools

- Ubuntu VM (Provided): Use the same preconfigured forensic environment from the TL.
- FAT32 Forensic Image (fat32_forensic.dd): Contains deleted and timestamp-altered files.
- Sleuth Kit CLI Tools (fls, fsstat, istat, icat): For forensic analysis and file recovery.
- Hex Editor (xxd): For manual verification of file headers.
- Python 3: Used for automated file carving.

Resources

- https://docs.google.com/document/d/1znfatFau0N6pg_LPBfcX_P9Cp4qxoD8CFatdib4KwZ4/copy: this will prompt you to make a copy.
- Python File Carving Script (file_carver.py) – Preloaded in VM : https://docs.google.com/document/d/1znfatFau0N6pg_LPBfcX_P9Cp4qxoD8CFatdib4KwZ4/edit?tab=t.0#heading=h.4wp63qfdlwm7

Task 1: Set Up the Virtual Environment

1. Launch the Ubuntu VM.

2. Verify Installed Tools.

Task 2: Verify the File System & Gather Metadata

1. Identify File System Type

- Mount the disk image (Read-Only Mode): sudo losetup -Pf fat32_forensic.dd

```
student@student-flatiron:~/Forensics$ sudo losetup -Pf fat32_forensic.dd
student@student-flatiron:~/Forensics$
```

- List partitions and file system information: sudo fsstat /dev/loop0

```
student@student-flatiron:~/Forensics$  sudo fsstat /dev/loop0
Possible encryption detected (High entropy (8.00))
student@student-flatiron:~/Forensics$
```

- Identify the file system format (e.g., NTFS, FAT32).

Note: if the fsstat command returns “cannot determine file system type”, check the file type using: file fat32_forensic.dd. You will see that this disk is FAT32.

```
student@student-flatiron:~/Forensics$ file fat32_forensic.dd
fat32_forensic.dd: DOS/MBR boot sector, code offset 0x58+2, OEM-ID "mkfs.fat", Media descriptor 0xf8, sectors/track 32,
 heads 8, sectors 102400 (volumes > 32 MB), FAT (32 bit), sectors/FAT 788, serial number 0xa742e8f9, unlabeled
student@student-flatiron:~/Forensics$
```

2. Gather File System Metadata

- Display file system metadata: fsstat fat32_forensic.dd
    - Key Information to Note:
        - File System Type (e.g., NTFS, FAT32)
        - Cluster Size (used for byte offset calculations)
- Expected Output (Example)
    - File System Type: FAT32
    - Cluster Size: 512 bytes
    - Write these down! They are essential for later steps.

SOLUTION

```
student@student-flatiron:~/Forensics$ fsstat fat32_forensic.dd
FILE SYSTEM INFORMATION
--------------------------------------------
File System Type: FAT32

OEM Name: mkfs.fat
Volume ID: 0xa742e8f9
Volume Label (Boot Sector): NO NAME
Volume Label (Root Directory):
File System Type Label: FAT32
Next Free Sector (FS Info): 64370
Free Sector Count (FS Info): 100791

Sectors before file system: 0

File System Layout (in sectors)
Total Range: 0 - 102399
* Reserved: 0 - 31
** Boot Sector: 0
** FS Info Sector: 1
** Backup Boot Sector: 6
* FAT 0: 32 - 819
* FAT 1: 820 - 1607
* Data Area: 1608 - 102399
** Cluster Area: 1608 - 102399
*** Root Directory: 1608 - 1608

METADATA INFORMATION
--------------------------------------------
Range: 2 - 1612678
Root Directory: 2
```

File System Metadata for FAT32:

- **File System Type**: FAT32
- **Cluster Size**: 512 bytes

Task 3: Locate Deleted File Information

1. List Files in the Disk Image

List all files, including deleted files (marked with -*): sudo fls -r fat32_forensic.dd

Look for deleted files.

Identify the entry number of a deleted file.

Example Output: r/r * 4: IMG_2019.jpg

SOLUTION

```
student@student-flatiron:~/Forensics$ sudo fls -r fat32_forensic.dd
r/r * 4:        IMG_2019.jpg
r/r * 6:        IMG_0117.png
v/v 1612675:    $MBR
v/v 1612676:    $FAT1
v/v 1612677:    $FAT2
V/V 1612678:    $OrphanFiles
```

- **IMG_2019.jpg** (Deleted)

- **IMG_0117.png** (Deleted)

These files are marked as deleted with an "r/r" flag and can be further analyzed using their respective entry numbers. For example:

- **IMG_2019.jpg**: Entry number 4
- **IMG_0117.png**: Entry number 6

2. Extract File Metadata & Calculate Byte Offsets
- Analyze the deleted file using its entry number: sudo istat fat32_forensic.dd 4
Note:
- File Creation, Accessed, & Written Timestamps
- File Size
- Sectors (needed for byte offsets)

Example Output (Key Sections)
Name: _MG_2019.jpg

Created: 2025-03-07 22:08:29

Written: 2036-01-01 07:59:58

Size: 3190011 bytes

Sectors: 32990 32991 32992 …
Why does this matter?
- Timestamps can indicate suspicious activity. The written timestamp for _MG_2019.jpg is 2036, which is significantly in the future. This could indicate data manipulation, an attempt to obfuscate evidence, or a system clock anomaly.
- Sectors are needed to calculate byte offsets for manual carving.

To carve the file, we need to calculate the byte offsets.
Start of File:
Sector Number * Cluster Size.
32990 * 512 = 16890880
Convert it to hexadecimal: printf "%x\n" 16890880
Hex value: 101bc00
End of File:
Start Of File + File Size
16890880 + 3190011 = 20080891
Hex value: 13268fb.
- Follow Steps 1 and 2 to gather metadata on the png file.

SOLUTION

```
student@student-flatiron:~/Forensics$ sudo istat fat32_forensic.dd 4
Directory Entry: 4
Not Allocated
File Attributes: File, Archive
Size: 3190011
Name: _MG_2019.JPG

Directory Entry Times:
Written:        2036-01-01 07:59:58 (PST)
Accessed:       2025-03-07 00:00:00 (PST)
Created:        2025-03-07 22:08:29 (PST)

Sectors:
32990 32991 32992 32993 32994 32995 32996 32997
32998 32999 33000 33001 33002 33003 33004 33005
33006 33007 33008 33009 33010 33011 33012 33013
33014 33015 33016 33017 33018 33019 33020 33021
33022 33023 33024 33025 33026 33027 33028 33029
33030 33031 33032 33033 33034 33035 33036 33037
33038 33039 33040 33041 33042 33043 33044 33045
33046 33047 33048 33049 33050 33051 33052 33053
33054 33055 33056 33057 33058 33059 33060 33061
33062 33063 33064 33065 33066 33067 33068 33069
33070 33071 33072 33073 33074 33075 33076 33077
33078 33079 33080 33081 33082 33083 33084 33085
33086 33087 33088 33089 33090 33091 33092 33093
33094 33095 33096 33097 33098 33099 33100 33101
33102 33103 33104 33105 33106 33107 33108 33109
33110 33111 33112 33113 33114 33115 33116 33117
33118 33119 33120 33121 33122 33123 33124 33125
```

File Metadata for IMG_2019.jpg:

- **Name: _MG_2019.JPG**
- **Created: 2025-03-07 22:08:29 (PST)**
- **Written: 2036-01-01 07:59:58 (PST)**
- **Accessed: 2025-03-07 00:00:00 (PST)**
- **Size: 3,190,011 bytes**
- **Sectors: 32990 - 33449 (Total: 460 sectors)**

Explanation:

1. **Timestamps:**
    - **Created: The file was created on 2025-03-07.**
    - **Written: The written timestamp is 2036-01-01, which is unusual and could suggest either manipulation or a system clock anomaly.**

- **Accessed: Accessed on 2025-03-07.**

2. **File Size: The file size is 3,190,011 bytes (approx. 3.19 MB).**

3. **Sectors: The file spans from sectors 32990 to 33449 (a range of 460 sectors).**

Byte Offset Calculation:

1. **Start of File:**

   - **Starting sector: 32990**

   - **Offset = Sector Number * Cluster Size = 32990 * 512 = 16,890,880 bytes (in decimal).**

   - **Hexadecimal: `0x101bc00`**

2. **End of File:**

   - **End sector: 33449**

   - **Offset = Start Offset + File Size = 16,890,880 + 3,190,011 = 20,080,891 bytes (in decimal).**

   - **Hexadecimal: `0x13268fb`**

File Metadata for IMG_0117.png:

Now, you can follow the same procedure to gather metadata and calculate byte offsets for the IMG_0117.png file, identified with entry number 6.

Task 4: Recover Deleted Files
Now that we have the start and end offsets, we can extract the file.

Method 1: Extract Deleted Files Using CLI Tools
1. Recover the file using its entry number:
sudo icat fat32_forensic.dd 4 > recovered_jpg.jpg

```
student@student-flatiron:~/Forensics$ sudo icat fat32_forensic.dd 4 > recovered_jpg.jpg
student@student-flatiron:~/Forensics$
```

Successfully recovered the deleted **IMG_2019.jpg** file using the `icat` command:

2. Verify file integrity using the file command:
file recovered_jpg.jpg

```
student@student-flatiron:~/Forensics$ file recovered_jpg.jpg
recovered_jpg.jpg: JPEG image data, JFIF standard 1.02, resolution (DPI), density 72x72, segment length 16, Exif Standa
rd: [TIFF image data, big-endian, direntries=14, manufacturer=Apple, model=iPhone 13 Pro, orientation=upper-left, xreso
lution=202, yresolution=210, resolutionunit=2, software=18.1.1, datetime=2025:02:17 12:28:59, hostcomputer=iPhone 13 Pr
o], baseline, precision 8, 3024x4032, components 3
student@student-flatiron:~/Forensics$
```

This confirms that the JPG file has been successfully recovered and is intact, with metadata indicating it was taken with an **iPhone 13 Pro**.

3. If the file is corrupted, analyze the file manually using a hex editor.
4. Recover the png file, naming it recovered_png.png

Method 2: Automate File Carving with Python

1. Create an offsets.txt file listing multiple SOF/EOF values. Each entry should be formatted as:
StartOffset, EndOffset, Filename
Example:
101bc00,13268fb,IMG_2019python.jpg
2. Run the script: python3 ~/Forensics/file_carver.py
- Enter the following when prompted:

- File name: fat32_forensic.dd
- Would you like to carve multiple files? (y/n): y
- Enter the offsets file name (e.g., offsets.txt): offsets.txt

```
student@student-flatiron:~/Forensics$ python3 ~/Forensics/file_carver.py

--- File Carving Script ---
Enter the forensic image file name (e.g., forensic_image.dd): fat32_forensic.dd
Would you like to carve multiple files? (y/n): y
Enter the offsets file name (e.g., offsets.txt): offsets.txt
Error: Offsets file not found.
```

An error occurred, let us try to solve it.
nano offsets.txt
101bc00,13268fb,IMG_2019.jpg
20234a0,212bb80,IMG_0117.png

```
student@student-flatiron:~/Forensics$ python3 ~/Forensics/file_carver.py

--- File Carving Script ---
Enter the forensic image file name (e.g., forensic_image.dd): fat32_forensic.dd
Would you like to carve multiple files? (y/n): y
Enter the offsets file name (e.g., offsets.txt): offsets.txt
File successfully carved and saved as:  IMG_2019.jpg
File successfully carved and saved as:  IMG_0117.png
student@student-flatiron:~/Forensics$
```

Task 5: Verify File Integrity (MD5 Hash Comparison)
Generate Hashes

1. Run the following commands to compute MD5 checksums:
md5sum recovered_jpg.jpg IMG_2019python.jpg
Expected Output (Example)
Original: b3ab520fa902f0f21d3e77d399aabde2 IMG_2019.jpg
Recovered: bfc40fd6b8adf2e7745d833de36b4bc0 recovered_jpg.jpg
Carved: b3ab520fa902f0f21d3e77d399aabde2 IMG_2019python.jpg

- **If the hashes match** the original, the file was successfully recovered!
- **If the hashes don't match**, check the byte offsets used in the carving process. (The recovered and carved should both match the original, which has been given above. The recovered showing a different MD5 hash is purely for demonstration purposes here. Both the Python script and the CLI method should carve an identical file).

2. Do the same with the png file.
SOLUTION
md5sum IMG_2019.jpg IMG_0117.png

```
student@student-flatiron:~/Forensics$ md5sum IMG_2019.jpg IMG_0117.png
b3ab520fa902f0f21d3e77d399aabde2  IMG_2019.jpg
8e58183fdd40b515a37c0d6e43cedcda  IMG_0117.png
student@student-flatiron:~/Forensics$
```